TIKO

Log. Watch the ring fill. Claim your treat.

TIKO is a calm web app (PWA) that helps people build better habits, spending and moods in under 30 seconds a day, and rewards every goal they reach with a treat they chose themselves.

It is built for students and working adults aged 18 to 35 who have tried habit or budgeting apps and quit. The design answers the reasons people quit: logging takes too many taps, one missed day breaks a streak, and nothing rewards sticking with it.

> **Status:** planning documents are complete; the MVP build (weeks 1 to 8) is next. Commands and folder names below describe the planned setup and should be adjusted to match `package.json` once the project is scaffolded.

What it does

In the MVP	Not in the MVP
Responsive web app (PWA) with guest mode and sign-in	Native iOS and Android apps (V2 wrapper)
Habits: 7 core plus custom, Today screen, daily ring	Automatic step and health-app import (V1)
Goals, treats, celebration screen, Trophy shelf	Savings goals with milestones, bundle goals (V1 and later)
Expense logging, monthly budget, safe-to-spend number	Bank linking or moving money (never)
Mood check-in and a simple encrypted journal	Social features, buddies, leaderboards (never)
Forgiving streaks, kind reminders, offline logging	Coach programmes, extra languages (V2)
Light and dark theme	Plus plan billing (V1)

The core loop. Open Today, tap to log, the goal engine runs, the ring grows (with a nudge at 50% and 90%), and when the target is met the celebration shows the treat you picked. Claim it, and it goes on the Trophy shelf. TIKO then offers to repeat the goal or raise it by about 10%.

Principles that shape the product

- A check-in takes under 30 seconds; the first log takes under 10 seconds from opening the app.

- Streaks are forgiving: one missed day in any 7 is forgiven, and nothing is ever marked as failed.
- Copy never shames: no "failed", "lazy" or "behind".
- Mind features are self-care, not therapy or medical advice, and never diagnose.
- TIKO never holds, moves or links to money, shows no ads, and does not sell data.

How it works

TIKO is **local-first**. Every screen reads and writes a database on the device, so a tap shows in 100 ms and logging works with no connection. A shared rules engine runs on both the device and the server, and the server backs up data, links devices and sends reminders.

- **One engine, two runtimes.** Goals, streaks, budget and XP live in a pure TypeScript package used by the browser and the server.
- **Derive, don't increment.** Progress is recomputed from logs, so undo, edits and offline merges stay correct.
- **Money is an integer** in the smallest unit (paise); the default currency is INR.
- **Conflicts:** the latest edit wins per row; nothing on the device is deleted because a sync failed.

See `docs/07-architecture.md` for the diagrams and reasoning.

Tech stack

These are proposals from the Technical Specification; the Tech Stack document wins if it differs.

Layer	Choice
App	Next.js (App Router), React, TypeScript
Styling	Tailwind CSS driven by design tokens
PWA	Service worker (Serwist), web manifest
Local store	IndexedDB through Dexie
API	Next.js route handlers, Zod validation
Database	PostgreSQL (Neon or Supabase), Drizzle ORM
Auth	Auth.js (email magic link and Google), or Supabase Auth with Supabase
Email and push	Resend; Web Push (VAPID)
Scheduler	External cron (for example Upstash QStash or Supabase pg_cron)
Monitoring	Sentry, uptime monitor
Tests	Vitest, fast-check, Playwright, Lighthouse CI, axe, k6
Hosting	Vercel
Payments (V1)	Razorpay

Getting started

Prerequisites

- Node.js (current LTS) and a package manager
- Docker, for a local PostgreSQL container
- Git

Setup

bash

```
git clone <repository-url> tiko
cd tiko
npm install

cp .env.example .env.local     # then fill in the values (see below)

docker compose up -d db        # local PostgreSQL
npm run db:migrate             # apply migrations
npm run dev                    # http://localhost:3000
```

Local email goes to a mail catcher (for example Mailpit) and web push to a stub, so no real person ever receives a message from a development build.

Scripts (planned)

Command	What it does
`npm run dev`	Start the app in development
`npm run build`	Production build
`npm run typecheck`	TypeScript check
`npm run lint`	Lint, including copy and hard-coded-string rules
`npm test`	Unit, property and integration tests
`npm run test:e2e`	Playwright end-to-end tests, including offline flows
`npm run db:migrate`	Apply database migrations
`npm run db:generate`	Generate a migration from schema changes

Environment variables

Copy `.env.example` to `.env.local`. **Never commit real secrets.** Only values prefixed `NEXT_PUBLIC_` reach the browser. Names are proposals and follow the chosen libraries.

Variable	Purpose
`AUTH_SECRET`	Signs session cookies
`DATABASE_URL`	App database connection (app role, no schema rights; pooled string in production)
`MIGRATION_DATABASE_URL`	Migration role; used only by the deploy step
`MASTER_ENCRYPTION_KEY`	Wraps each user's journal key. Back it up offline; losing it makes journals unreadable
`RESEND_API_KEY`, `EMAIL_FROM`	Magic links and fallback reminders
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	Google sign-in
`NEXT_PUBLIC_VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, `VAPID_SUBJECT`	Web push signing. Do not rotate casually: it invalidates every subscription
`CRON_SECRET`	Authenticates scheduled-job calls
`RATE_LIMIT_REDIS_URL`, `RATE_LIMIT_REDIS_TOKEN`	Sign-in and write rate limits
`BLOB_READ_WRITE_TOKEN`	Treat photo storage (or the S3-compatible equivalent)
`SENTRY_DSN`	Error monitoring, with personal content scrubbed
`ANALYTICS_KEY`	Product events through the allowlist
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`	Plus plan billing (V1)

Project structure (planned)

```
tiko/
├── app/                    Next.js routes: screens and /api route handlers
├── packages/
│   └── engine/             Pure TypeScript rules: goals, streaks, budget, XP
(no I/O)
├── src/
│   ├── db/                 Dexie schema, entity types, outbox
│   ├── sync/               Push, pull, merge-guest, conflict handling
│   ├── features/           habits, goals, money, mind, me
│   ├── ui/                 Design-token components, theme, accessibility
helpers
│   └── platform/           Service worker, push, install, storage persistence
├── drizzle/                Server database migrations
├── public/                 Web manifest, icons
├── tests/                  End-to-end and scenario tests
├── docs/                   Planning and design documents (below)
├── .env.example
├── README.md
├── package.json
└── tsconfig.json
```

Rule of thumb: screens call the domain layer, the domain layer calls the local store, and only the sync and platform code touches the network. A screen never calls `fetch` directly.

Testing

Most checks are automated and run on every pull request: typecheck, lint, unit and property tests for the engine, integration tests against a real Postgres, smoke end-to-end flows, an axe accessibility scan and a Lighthouse performance budget. Nightly runs add the full cross-browser suite, offline and sync scenarios, a load test and security scans.

Anything that depends on the date (streaks, deadlines, budgets, quiet hours, link expiry) runs on a controlled clock and in several time zones, including Asia/Kolkata and UTC. The full plan, with expected values for the engine, is in `docs/10-testing.md`.

Deployment

Every change goes through the same pipeline: pull request checks, a preview deploy, merge to main, staging deploy and smoke test, a manual approval, then production. Database migrations run before the new code starts and are written so the previous release keeps working. Details, rollback steps and the launch checklist are in `docs/11-deployment.md`.

Documentation

All planning documents live in `docs/`. Where a detailed document differs from a summary, the detailed one wins.

File	What it holds
`docs/01-project-brief.md`	Vision, users, scope, success measures
`docs/02-prd.md`	Requirements, non-functional targets, release criteria
`docs/03-user-stories.md`	Personas, stories, acceptance criteria, use cases
`docs/04-functional-requirements.md`	Numbered requirements (FR-IDs) by area with priority
`docs/05-ui-ux.md`	Colors, type, components, screens, tokens
`docs/06-technical-spec.md`	Build decisions, engines, offline sync, build order
`docs/07-architecture.md`	System design, flows, trust boundaries
`docs/08-database-design.md`	Schema, constraints, indexes, retention
`docs/09-api.md`	Routes, payloads, errors, rate limits
`docs/10-testing.md`	Test levels, acceptance tests, engine cases, gates
`docs/11-deployment.md`	Environments, pipeline, secrets, backups, rollback

Roadmap

Release	Weeks	Adds
MVP	1 to 8	The log, goal, treat loop; money; mind; offline; reminders
V1	9 to 16	Savings goals, weekly recap, health-app step import, Plus plan
V2	17 to 26	Native wrapper, more languages, coach-led challenges

Each release starts only after its gate is met: 10 beta users logging 4 or more days a week before V1 work, and day-30 retention near 20% before V2.

Contributing

TIKO starts with one developer, so these conventions keep the code consistent:

- Put rules in `packages/engine` as pure functions with tests; do not copy rules into screens or routes.
- Store money as integer paise; never use floats for money.
- Write the acceptance test for a P0 requirement before building it.
- Never write copy that shames, diagnoses, or gives medical, diet or investment advice.
- Keep journal text, mood notes and exact amounts out of analytics, notifications, logs and error reports.
- Add a regression test with every fix to engine, sync or privacy code.

Privacy and safety

Journal bodies are encrypted at rest on the server. This is not end-to-end encryption, and the local copy on a device is not encrypted by the app. Export and account deletion are built in, and deleted data is purged within 30 days. TIKO is a self-care tool and is not therapy or medical advice. A review against India's DPDP Act 2023 and GDPR is required before public launch.

License

To be decided before the repository is made public.